

# CodeShield AI

## System Architecture & Data Flow

This document details the software architecture, file connections, and multi-agent orchestration workflows within CodeShield AI.

### 1. High-Level Architecture Diagram

The diagram below represents how different layers of CodeShield AI (Frontend, FastAPI Backend, Scanner Engine, Multi-Agent Swarm, AI engines, and Database) are connected and pass data.

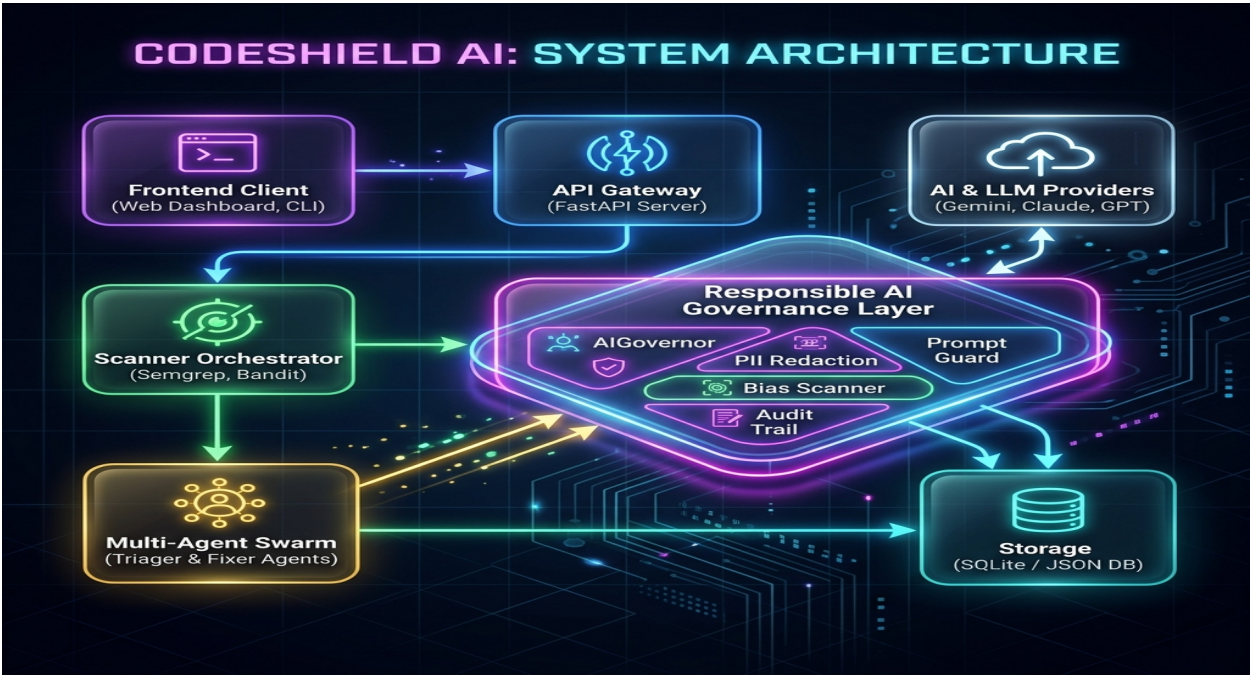


Figure: CodeShield AI System Architecture Map

```
graph TD
  %% Frontend Layer
  subgraph Frontend [Static UI / Clients]
    UI[static/index.html & app.js]
    CLI[cli.py]
    CICD[cicd/ Generators]
  end

  %% Web & Controller Layer
  subgraph Controller [Web & API Layer]
    Main[main.py - FastAPI Application]
  end
```

```

    Config[utils/config.py - Settings]
end

%% Scanner Engine
subgraph Scanning [Scanner Infrastructure]
    Engine[scanner/engine.py]
    Scanners[scanner/tools/ - bandit, semgrep, gitleaks, eslint]
end

%% Agentic Swarm Layer
subgraph Swarm [Multi-Agent Swarm / CrewAI]
    Registry[agents/registry.py - Lifecycle]
    Workflows[agents/workflows.py]
    TriagerAgent[agents/triager.py]
    FixAgent[agents/fix_agent.py]
    ReportAssembler[agents/report_assembler.py]
end

%% AI Engines
subgraph AIEngines [AI/LLM Core Processing]
    TriageEngine[ai_triage.py - Heuristics + LLM Cascade]
    FixEngine[auto_fix.py - Regex + LLM Cascade]
end

%% Governance Layer
subgraph Governance [Responsible AI Governance Layer]
    Assist[governance/assist.py - governed_complete]
    Governor[governance/governor.py - AIGovernor]
    Policy[governance/policy.py - ResponsibleAIPolicy]
    PromptGuard[governance/prompt_guard.py - PromptGuard]
    PIIRedactor[governance/pii.py - PIIRedactor]
    BiasScanner[governance/bias.py - BiasScanner]
    AuditTrail[governance/audit.py - AuditTrail]
end

%% LLM Providers
subgraph LLMPProviders [AI & LLM Providers]
    LLM[llm/ - LLM Cascade Providers]
end

%% Storage
subgraph Database [Storage & Models]
    DB[database/json_db.py]
    Models[models/vulnerability.py - Vulnerability & ScanResult Schema]
end

%% Connections
UI -->|REST APIs / WebSocket| Main
CLI -->|REST APIs| Main
CICD -->|Triggers Scan via| Main

Main -->|Loads| Config
Main -->|Orchestrates Scans| Engine
Engine -->|Runs SAST/DAST| Scanners
Scanners -->|Outputs Raw Findings| Engine

Engine -->|Serializes into Vulnerability Models| Models
Main -->|Invokes Swarm Workflows| Workflows

Workflows -->|Queries Agent Registry| Registry
Registry -->|Coordinates Agents| TriagerAgent
Registry -->|Coordinates Agents| FixAgent
Registry -->|Coordinates Agents| ReportAssembler

TriagerAgent -->|Delegates Analysis| TriageEngine
FixAgent -->|Delegates Remediation| FixEngine

```

```
TriageEngine -->|Calls governed_complete| Assist
FixEngine -->|Calls governed_complete| Assist

Assist -->|Queries AIGovernor| Governor
Governor -->|Validates Policy| Policy
Governor -->|Redacts PII| PIIRedactor
Governor -->|Screens Prompts| PromptGuard
Governor -->|Scans Bias| BiasScanner
Governor -->|Records Action| AuditTrail

Governor -->|Forwards Safe Prompt| LLM
LLM -->|Returns Raw Response| Governor

TriageEngine -->|Saves Verdict & State| DB
FixEngine -->|Saves Fixes & Diff State| DB
AuditTrail -->|Writes Hashes & Traces| DB
DB -->|Reads/Writes Scan Records| Models
```

---

## 2. Component Directory Structure & Key Files

Here is how the project files are structurally organized and connected:

```
ai-code-sheild/
■■■■ main.py                # Entrypoint. FastAPI Server routing APIs (scan, fix, triage, download)
■■■■ ai_triage.py           # AI triage engine. Heuristics + LLM cascade (OpenAI, Gemini, Ollama)
■■■■ auto_fix.py           # AI auto-remediation engine (Regex-based + LLM generation, validation & diffs)
■■■■ cli.py                # CLI tool allowing users to trigger scans and generate reports from terminal
■■
■■■■ static/               # Frontend files served at '/'
■■   ■■■■ index.html       # Premium glassmorphic Dashboard HTML
■■   ■■■■ styles.css       # Styling for the UI, diff grids, and animations
■■   ■■■■ app.js           # JS router making API requests to main.py
■■
■■■■ agents/              # Multi-Agent Swarm (CrewAI based)
■■   ■■■■ registry.py      # Dynamic registration, health, and status of active agents
■■   ■■■■ workflows.py     # Orchestrates agent handoffs (Triage -> Fix -> Report)
■■   ■■■■ triager.py       # Agent wrapper invoking ai_triage.py logic
■■   ■■■■ fix_agent.py     # Agent wrapper invoking auto_fix.py logic
■■
■■■■ scanner/             # Code scanning modules
■■   ■■■■ engine.py        # Orchestrator initializing and running parallel scans
■■   ■■■■ tools/           # Tool runners wrapping CLI scanners
■■       ■■■■ bandit_scanner.py # Python SAST scanner
■■       ■■■■ semgrep_scanner.py # Multilanguage SAST
■■       ■■■■ gitleaks_scanner.py # Secret detection
■■
■■■■ cicd/                # CI/CD Pipeline Generators
■■   ■■■■ github_action.py  # Outputs GitHub workflow configurations
■■   ■■■■ jenkins_plugin.py  # Outputs Jenkins scripted and declarative pipeline Groovy code
■■
■■■■ database/            # File-based JSON database engine simulating scan histories
■■   ■■■■ json_db.py
■■
■■■■ models/              # Pydantic schemas (Vulnerability, ScanResult, is_fixed state)
■■   ■■■■ vulnerability.py
```

---

## 3. End-to-End Execution Flow

When a user initiates a scan, the data flows step-by-step through the files as follows:

```
sequenceDiagram
    autonumber
```

```

actor User as Developer (UI/CLI/CI)
participant Server as main.py (FastAPI)
participant Scanners as scanner/engine.py
participant DB as database/json_db.py
participant Swarm as agents/workflows.py
participant Triage as ai_triage.py
participant Gov as governance/governor.py (AIGovernor)
participant LLM as llm (Providers)
participant Fixer as auto_fix.py

User->>Server: Upload codebase (ZIP or GitHub URL)
Server->>Server: Unpack archive to tmp/<scan_id>/
Server->>Scanners: Trigger scan engine
Scanners->>Scanners: Run parallel scanners (Semgrep, Bandit, etc.)
Scanners-->>Server: Return raw vulnerability schemas
Server->>DB: Save initial scan results (pending triage)

Server->>Swarm: Dispatch Swarm Triage Workflow
Swarm->>Triage: Run triage heuristics & AI cascades

Triage->>Gov: governed_complete(prompt)
Gov->>Gov: Inspect Prompt (PromptGuard) & Redact PII (PIIRedactor)
Gov->>LLM: Forward Sanitized Prompt
LLM-->>Gov: Return Raw Response
Gov->>Gov: Scan Response for Bias (BiasScanner)
Gov->>DB: Append Session Hash & Trace (AuditTrail)
Gov-->>Triage: Return Inspected & Sanitized Response

Triage-->>Swarm: Adjust confidence (HIGH/MEDIUM) or flag False Positives
Swarm->>DB: Save triaged scan findings

User->>Server: Click "Preview Auto-Fix" / Fix API
Server->>Fixer: Request remediation for vulnerability ID

Fixer->>Gov: governed_complete(patch_prompt)
Gov->>Gov: Inspect Prompt (PromptGuard) & Redact PII (PIIRedactor)
Gov->>LLM: Forward Sanitized Prompt
LLM-->>Gov: Return Raw Response
Gov->>Gov: Scan Response for Bias (BiasScanner)
Gov->>DB: Append Session Hash & Trace (AuditTrail)
Gov-->>Fixer: Return Remediation Code Patch

Fixer->>Fixer: Validate patched code syntax & verify fix
Fixer-->>Server: Return unified Diff structure
Server-->>User: Display Diff visual layout in UI

User->>Server: Click "Apply Fix" / Apply API
Server->>Fixer: Write patched file to tmp/<scan_id>/source/...
Server->>DB: Mark vulnerability is_fixed = True
Server-->>User: Show success badge on UI card

User->>Server: Click "Download Patched Code"
Server->>Server: Compress tmp/<scan_id>/ base directory to ZIP
Server-->>User: Deliver ZIP file response

```

---

## 4. Multi-Agent Swarm Orchestration Mechanics

### The Swarm Lifecycle (`agents/registry.py` & `ai\_team/coordinator.py`)

1. **Registration:** When agents start up (using `registry.register\_agent`), they publish their `AgentCapabilities` (what tools they own, what programming languages they can write code in, and which vulnerability types they target).

2. **Heartbeats:** Registry monitors dynamic agent health (`HEALTHY`, `BUSY`, `FAILED`) to perform load balancing during high concurrency.
3. **Event Listeners & Orchestration:** The orchestrator listens for status changes to execute workflows asynchronously. The `Coordinator` coordinates agent turns and passes context between them.

## The Agent Swarm Workflow (`agents/workflows.py`)

Instead of running monolithic scans, the work is divided into agent roles:

- **`JohnSAST` / `TinaTaint` (Security Engineers):** Run semantic and static taint analysis to detect flows of untrusted inputs.
- **`TriagerAgent` (Governance Auditor):** Uses `ai_triage.py` to examine the context of security alerts and filter out noise (e.g. flagging code located in test folders as a False Positive).
- **`FixAgent` (Remediation Engineer):** Uses `auto_fix.py` to draft syntactically sound code patches, validate security changes, and produce the diff.
- **`ReportAssembler` (Compliance Writer):** Aggregates findings and generates JSON, HTML, SARIF, or PDF formats for the user.

## Governance Shielding on Agent Actions (`ai_team/agent.py`)

Rather than communicating directly with external models, each agent (`TeamAgent`) interacts through the `Responsible AI Governance Layer`:

1. **Request Interception:** On calling `run()`, the agent builds a prompt and queries `AIGovernor.ask()` with its dynamic `RoleSpec` system prompt and data `sensitivity` level.
2. **PII and Secret Protection:** The `PIIRedactor` censors sensitive artifacts (e.g. passwords, hardcoded keys, API keys, developer emails) in the input/output codebase contexts.
3. **Prompt Injection Prevention:** The `PromptGuard` evaluates prompts against adversarial inputs before forwarding them to `llm/` providers.
4. **Bias and Policy Scanning:** On LLM return, `BiasScanner` validates output safety. Any policy violation triggers a `GovernanceError`, blocking execution.
5. **Traceability:** Step outcomes report a `requires_human_review` status and map structural trace logs.
6. **Cumulative Governance Rollup:** The `Coordinator` aggregates all individual step traces into a central scan audit log (`governance_rollup()`) for compliance verification.